КУРС «МАШИННОЕ ОБУЧЕНИЕ И АНАЛИЗ ДАННЫХ»

Линейные модели и градиентный спуск

От нейрона МакКаллока-Питтса и перцептрона Розенблатта — к сигмоиде, методу наименьших квадратов и итеративному обучению.

90 минут | 5 модулей | Лекция 4 из 24

РОУДМАП

План на 90 минут

От имитации биологического нейрона до промышленного SGDRegressor

20 МИН

Исторические основы

Нейрон МакКаллока-Питтса и правило Хебба

20 МИН

От Перцептрона к Сигмоиде

Линейный классификатор и логистическая регрессия

15 МИН

Линейная регрессия и МНК

Аналитическое решение — нормальное уравнение

20 МИН

Градиентный спуск

Итеративное обучение вместо обращения матриц

15 МИН

Виды GD и практика

Batch, SGD, Mini-batch, код на Python

ПОЧЕМУ ЛИНЕЙНЫЕ МОДЕЛИ ПРАВЯТ МИРОМ

От запоминания данных к сжатию знаний

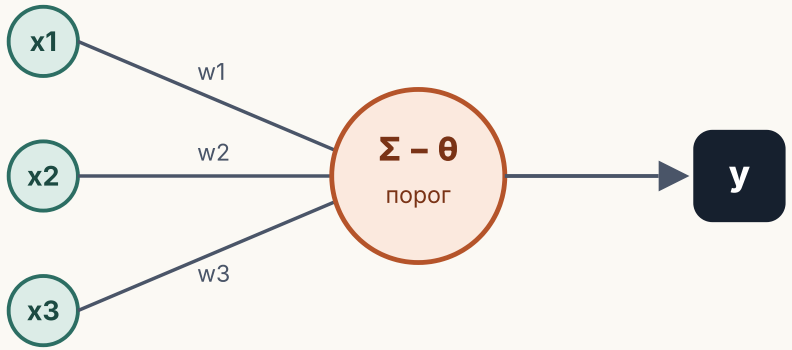
kNN не строит модель — он просто запоминает. В эпоху Big Data это непозволительная роскошь

- Линейные модели «сжимают» все знания о данных в компактный набор чисел (параметров)
- До сих пор используются в банках для скоринга и в медицине для диагностики
- Являются фундаментальным строительным блоком любой современной нейронной сети

1943 ГОД

Нейрон МакКаллока-Питтса

Идея машинного обучения родилась не из математики, а из нейрофизиологии



Нейрон либо передаёт импульс дальше, либо молчит — принцип «всё или ничего». f(z) выдаёт 1, если z ≥ 0 (функция Хевисайда).

$$y = f\left(\sum_{i=1}^{n} w_i x_i - \theta\right)$$

ПРИЁМ ЭТОЙ ЛЕКЦИИ

Строгий охранник в клубе

КЛУБ ЛЮБИТЕЛЕЙ БАНАНОВ

Биологический нейрон работает как строгий охранник в клубе любителей бананов. У него есть порог (например, 100 баллов). К нему подходят признаки банана: «Жёлтый цвет» даёт +50 баллов, «Сладкий запах» даёт +60 баллов, «Твёрдый как камень» даёт −20 баллов (штраф). Охранник суммирует: 50+60−20=90 баллов. Порог не пройден! Охранник молчит (выдаёт 0), банан в клуб не проходит.

ФАКТ

Уолтер Питтс был гением-самоучкой. В 12 лет он сбежал из дома, три дня прятался в библиотеке Чикагского университета, читая «Principia Mathematica» Рассела, нашёл там ошибки и написал автору письмо. Рассел пригласил его в аспирантуру в Кембридж, не зная, что общается с бездомным 12-летним мальчиком!

1949 ГОД

Правило Хебба: как заставить нейрон учиться

«Нейроны, которые активируются вместе, связываются вместе»

$$w_{new} = w_{old} + \eta \cdot y \cdot x$$

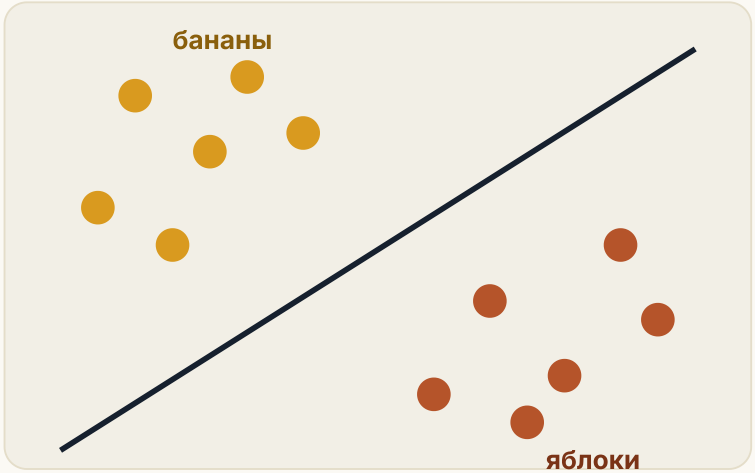
ДРЕССИРОВКА ЩЕНКА

Ты говоришь «Голос!» (входной сигнал x) и показываешь ему банан (награда y). Если щенок лает, ты даёшь ему банан. В его мозге связь между командой «Голос» и ожиданием банана становится всё толще и крепче. В следующий раз он залает намного увереннее! Это была первая в мире концепция итеративного обучения машины.

1958 ГОД

Перцептрон Розенблатта

Ищет гиперплоскость, которая идеально разделяет объекты двух классов



$$w_{new} = w_{old} + \eta \cdot (y_{true} - y_{pred}) \cdot x$$

Если перцептрон ответил верно, скобка равна нулю — вес НЕ меняется. Алгоритм корректирует себя только на ошибках!

ПРИЁМ ЭТОЙ ЛЕКЦИИ

Деревянная палка на столе

ЛИНЕЙНАЯ РАЗДЕЛИМОСТЬ

Представь, что на столе лежит куча жёлтых бананов и зелёных яблок, перемешанных между собой. Линейный классификатор — это жёсткая, прямая деревянная палка. Ты кладёшь её на стол так, чтобы все бананы оказались по одну сторону от палки, а все яблоки — по другую. Если тебе это удалось, значит твои фрукты «линейно разделимы»!

ФАКТ

Розенблатт реализовал перцептрон не в виде кода, а «в железе»! Компьютер Mark I Perceptron состоял из 400 фотоэлементов, моторов и потенциометров, которые физически крутились, изменяя сопротивление тока. Он был размером с огромный шкаф и умел распознавать буквы алфавита.

ГЛАДКАЯ АЛЬТЕРНАТИВА СТУПЕНЬКЕ

Проблема ступеньки и появление Сигмоиды

Производная ступеньки везде равна нулю — методы градиентов неприменимы



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

σ(z) интерпретируется как вероятность P(y=1|x). Замена ступеньки на сигмоиду превращает линейный классификатор в **Логистическую регрессию**.

ПРИЁМ ЭТОЙ ЛЕКЦИИ

Жёсткий человек и сомневающийся философ

СТУПЕНЬКА VS СИГМОИДА

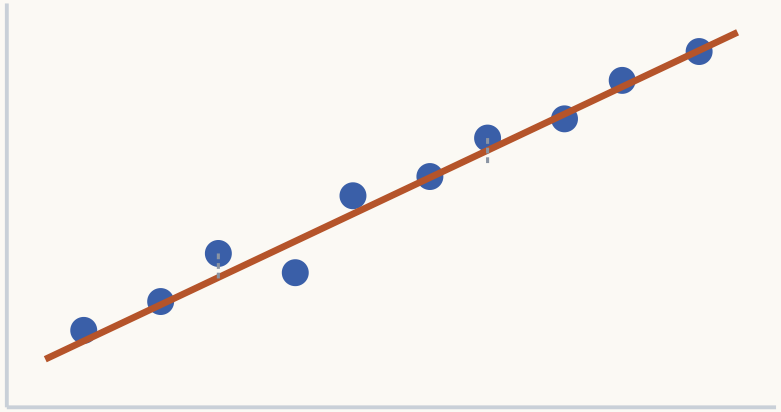
Ступенчатая функция — это жёсткий человек. Он смотрит на картинку и говорит: «Это банан» (1) или «Это не банан» (0). И никаких компромиссов! Сигмоида — это сомневающийся философ. Он смотрит на изогнутый жёлтый предмет и говорит: «Хм... на 85% я уверен, что это банан, но есть 15% вероятности, что это просто странная жёлтая сосиска».

ФАКТ

Сигмоидальная функция исторически возникла не в ИИ, а в демографии! В 1838 году математик Пьер Франсуа Ферхюльст вывел эту кривую для описания роста численности населения: сначала рост экспоненциальный, но затем он замедляется из-за нехватки ресурсов, выходя на плато.

ПРЕДСКАЗАНИЕ НЕПРЕРЫВНЫХ ЧИСЕЛ

Уравнение линейной регрессии



признак x (площадь квартиры)

$$\hat{y} = w_0 + w_1 x_1 + \ldots + w_n x_n = w^T x$$

Функция потерь — MSE:

$$L(w) = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2 = \frac{1}{N}\|Xw - y\|^2$$

АНАЛИТИЧЕСКОЕ РЕШЕНИЕ

Метод наименьших квадратов (МНК)

Взяли производную, приравняли к нулю — и сразу получили точный ответ

$$w = (X^T X)^{-1} X^T y$$

МАГИЧЕСКИЙ ТРАНСПОРТИР

Мы нарисовали на стене точки (рост и вес разных людей) и хотим провести через них идеальную прямую. Аналитическое решение — это магический транспортир: вбиваешь координаты точек, и он моментально выдаёт точный угол наклона. Никаких попыток и ошибок, сразу идеальный ответ.

ФАКТ

МНК придумал Карл Фридрих Гаусс в 1795 году, когда ему было всего 18 лет! Позже он использовал метод, чтобы по нескольким точкам на небе предсказать орбиту карликовой планеты Цереры.

ИТЕРАТИВНОЕ ОБУЧЕНИЕ

Суть градиентного спуска



- Градиент ∇L(w) указывает направление **наискорейшего возрастания**
- Антиградиент (−∇L(w)) указывает туда, где функция убывает быстрее всего
- Шаг за шагом мы спускаемся к минимуму функции потерь

$$w_{t+1} = w_t - \eta \cdot \nabla L(w_t)$$

ПРИЁМ ЭТОЙ ЛЕКЦИИ

Спуск с завязанными глазами

ГОРА БАНАНОВ В ДОЛИНЕ

Ты стоишь на вершине горы с плотно завязанными глазами, и тебе нужно найти самую низкую точку в долине, где лежит гора бананов. Ты нащупываешь ногами землю: слева земля уходит вверх, а справа — резко вниз (это и есть вычисление градиента). Делаешь один шаг вправо, вниз (антиградиент). Снова щупаешь землю. И так, шаг за шагом, ты спускаешься прямо к бананам! Это и есть Градиентный спуск.

ФАКТ

Градиентный спуск предложил Огюстен Луи Коши ещё в 1847 году — для решения систем уравнений при расчёте орбит небесных тел в астрономии. Математика XIX века стала двигателем нейросетей XXI века!

ЦЕНА ТОЧНОСТИ

Почему не всегда аналитика?

Обращение матрицы $(X^TX)^{-1}$ стоит $O(d^3)$, где d — число признаков

10 признаков

$10^3 = 1\,000$

операций — мгновенно.

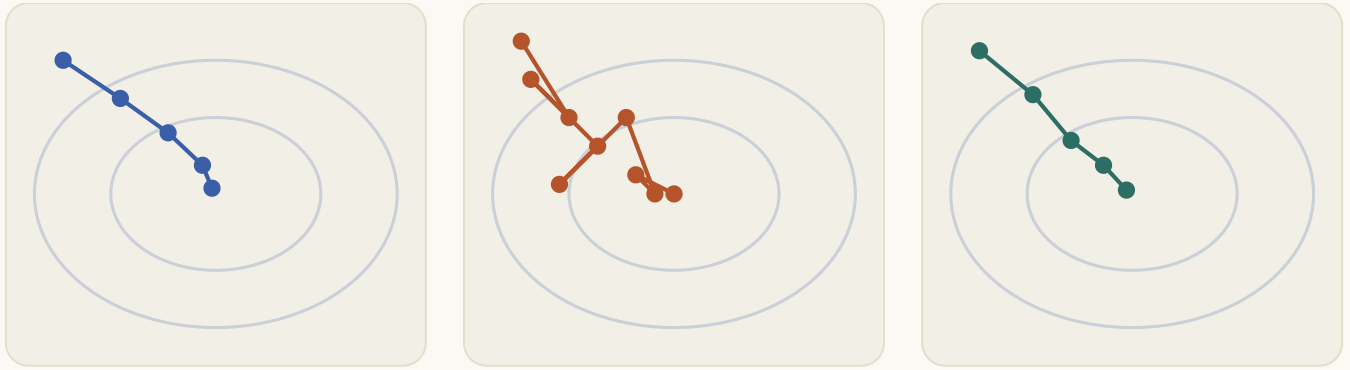
100 000 признаков (пиксели)

$(10^5)^3 = 10^{15}$

операций — квадриллион, годы расчётов!

РАЗНОВИДНОСТИ ГРАДИЕНТНОГО СПУСКА

Batch, Stochastic, Mini-batch



Batch GD — стабильно, медленно

SGD — быстро, шумно

Mini-batch — золотая середина

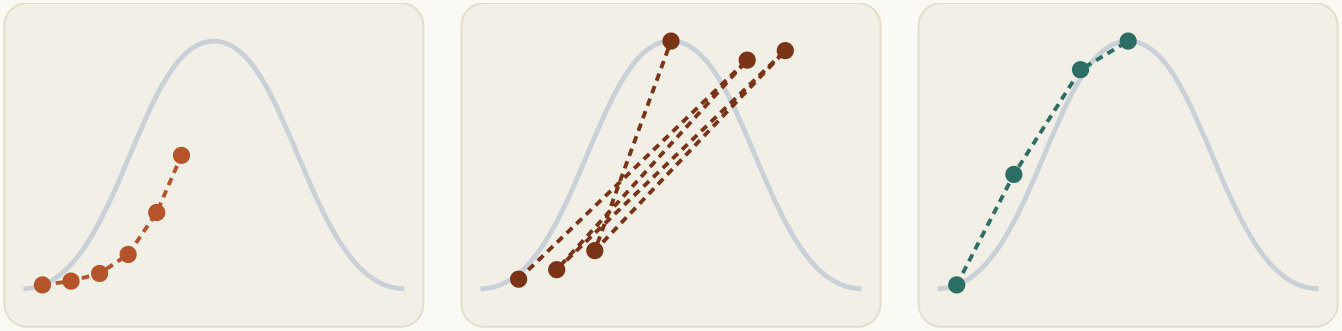
ПРИЁМ ЭТОЙ ЛЕКЦИИ

Кого спросить, куда идти вниз?

- **Batch (Пакетный):** Ты просишь 1 миллион человек проголосовать, куда идёт склон горы. Ждёшь 3 года, пока они ответят, и делаешь 1 идеальный шаг.
- **SGD (Стохастический):** Ты хватаешь одного случайного прохожего за рукав и спрашиваешь: «Где низ?». Он говорит «Туда!». Ты бежишь. Но прохожий мог быть пьян! Тебя мотает из стороны в сторону, но ты быстро движешься.
- **Mini-batch:** Ты берёшь 32 надёжных друга. Вы быстро совещаетесь за секунду и делаете шаг. Быстро, надёжно и не укачивает!

СКОРОСТЬ ОБУЧЕНИЯ (LEARNING RATE H)

От η зависит, сойдётся ли алгоритм



Маленький LR — очень медленно

Большой LR — расходится

Оптимальный LR — быстро сходится

ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ

LinearRegression vs SGDRegressor

```
# 100 000 объектов, 50 признаков — сравним скорость
X, y = make_regression(n_samples=100000, n_features=50, noise=10.0)
X_scaled = StandardScaler().fit_transform(X)

# Аналитическое решение (SVD под капотом)
model_ols = LinearRegression()
model_ols.fit(X_scaled, y) # медленно на многих признаках

# Итеративный градиентный спуск
model_sgd = SGDRegressor(max_iter=1000, eta0=0.01, learning_rate='adaptive')
model_sgd.fit(X_scaled, y) # летает даже на больших данных!
```

РЕЗЮМЕ

Пять тезисов сегодняшней лекции

01 **От нейрофизиологии к алгоритмам**
МакКаллок-Питтс (1943) и правило Хебба (1949) заложили идею обучаемого нейрона.

02 **Перцептрон и Сигмоида**
Замена жёсткой ступеньки на гладкую сигмоиду дала Логистическую регрессию.

03 **МНК — точный, но дорогой**
Нормальное уравнение $w=(X^TX)^{-1}X^TY$ даёт идеальный ответ за одно действие.

04 **Градиентный спуск — альтернатива**
Итеративный спуск по антиградиенту работает там, где обращение матрицы невозможно.

05 **Batch, SGD, Mini-batch**
Выбор разновидности GD и learning rate определяет скорость и стабильность обучения.

КОНТРОЛЬНЫЕ ВОПРОСЫ

5 вопросов для самопроверки

- Чем правило Хебба отличается от правила обновления весов Розенблатта?
- Почему ступенчатую функцию активации заменили на сигмоиду?
- В чём смысл нормального уравнения и когда его невыгодно использовать?
- Что показывает антиградиент функции потерь?
- Чем Mini-batch градиентный спуск лучше чистого Batch и чистого SGD?

ПРАКТИКА

Домашнее задание

01
Данные
Загрузить housing.csv, масштабировать признаки StandardScaler.

02
Модели
Обучить LinearRegression и SGDRegressor, сравнить MSE и время обучения.

03
Learning Rate
Поэкспериментировать с eta0 в SGDRegressor: показать расхождение при слишком большом значении.

Спасибо за внимание

Курс «Машинное обучение и анализ данных» · Лекция 4 из 24

Следующая лекция: Регуляризация и полиномиальные признаки